ARK-492 — Evidence Engine · Cold Start Latency

Status: PREREGISTRATION (locked before execution)
Experiment Date: 2026-07-18
Series: ExecutionProof P02 Latency/Throughput/Scale

Question

What is the cold-start latency — time from engine construction to first correct evidence verification — of the reference **Evidence Engine**?

The Evidence Engine answers the EVIDENCE half of Verification-Before-Execution: “Is there a complete, tamper-evident record proving the execution matched what was authorized — one that cannot be silently altered after the fact?” This completes the P02 component coverage: Verification Decision (ARK-483–486), Authority Engine (ARK-487–491), Evidence Engine (ARK-492).

Scope & Honesty Boundary

The component under test is a **deliberately minimal, in-process reference implementation** (`engine/evidence_engine.py`) built to MEASURE component performance. It is **NOT** the production Evidence Engine. A production engine (durable append-only store, replication, external notarization/timestamping) is out of scope for these testbeds and belongs to a governed customer integration. All claims are bounded to this in-memory reference under the stated load.

Hypothesis

Cold start (construct reference evidence chain of 10,000 records, then verify the first record) completes with **p95 ≤ 100,000 µs (100 ms)**. Higher than the Authority Engine (ARK-487) because chain construction computes 10,000 SHA-256 hashes.

Component Under Test

Reference EvidenceEngine — hash-chained, tamper-evident proof records:
1. each record binds (principal, action, authorization_ref, outcome)
2. `entry_hash = sha256(prev_hash || canonical(record))`
3. `verify_record(i)` recomputes the hash (tamper check) AND checks the chain link to the predecessor (chain check)

ALLOW only if the record is intact AND correctly chained. Exact, fail-closed. No normalization.

- **Implementation:** Python (`engine/evidence_engine.py`)
- **Harness:** `engine/perf_harness.py --dimension cold_start`

Methodology

Test Design

- **Runs:** 200 independent cold-start cycles
- Each run: construct a fresh reference chain (10,000 records) → verify the first record → record elapsed time (μs)
- **Correctness gate:** an intact record must ALLOW; a content-tampered record must DENY; a broken-chain record must DENY

Metrics

Primary: `cold_start_us` — mean, median, p95 (μs)
Secondary: min, max

Thresholds (Gate Conditions)

C1 (Latency): p95 cold start ≤ 100,000 μs
C2 (Correctness): correctness gate must PASS (intact→ALLOW, tampered→DENY, broken chain→DENY)

Verdict: PASS if C1 ∧ C2; otherwise FAIL

Kill-Gate

K1: If a tampered or broken-chain record verifies as ALLOW → the tamper-evidence property is void → GATE-STOP; latency numbers void.

Honest Findings Commitment

Any deviation from hypothesis (higher/lower than predicted, correctness failures) will be disclosed in RESULTS.md. If the correctness gate fails, the experiment is reported as GATE-STOP regardless of latency.

Execution Protocol

1. **Lock:** Commit this preregistration + compute MANIFEST.txt hashes
2. **Execute:** Run `engine/perf_harness.py --dimension cold_start`
3. **Record:** Save results to `results/coldstart_results.json`
4. **Evaluate:** Compare against thresholds C1, C2
5. **Report:** Document findings in RESULTS.md with honesty

Preregistered: 2026-07-18
Investigator: Remnant Fieldworks Inc. / ExecutionProof Research Program
Covenant: RF Standing Covenant (preserve outcomes, bound claims, honest disclosure)